

Apr 22, 26 13:19

main.cpp

Page 1/7

```

#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <avr/interrupt.h>

#define bitSet(reg, n)    ((reg) |= (1 << (n)))
#define bitClear(reg, n) ((reg) &= ~(1 << (n)))
#define bitCheck(reg, n) ((reg) >> (n)) & 1

#define start_tc0        (TCCR0B |= 0b001)    // prescaler = 1
#define stop_tc0         (TCCR0B &= ~0b111)

// Timer/Counter overflow interrupt enable and disable
#define enable_tc0_int    (bitSet(TIMSK0, TOIE0))
#define disable_tc0_int   (bitClear(TIMSK0, TOIE0))

#define pin_trigger PD4
#define pin_echo PD7
#define pin_oc0b PD5 // Changed from PD6 because OCR0A is now TOP

#define prescaler 1

// Testing TOP values. Initially anything less than 245 would hang
// the program in my_delay_us(). This was due to a counter overflow
// fov numOV in the routine.
//
// #define MY_TOP 219
// #define MY_TOP 245
// #define MY_TOP 199
// #define MY_TOP 240

// Total_time_ taken for the counter to count from 0->TOP-1->0 again.
float Tov;

// T=1/F, the period of each clock tick. Where is F/prescaler.
float clock_tc0;

// Kai's original code, causes my_delay_us() to hang as numOV overflows:
//volatile unsigned int numOV = 0;
//volatile unsigned int numOV_max_sonar = 0;
//
// Avoids counter overflow in my_delay_us() for 1s delay.
volatile uint32_t numOV = 0;

// Max number of overflows the sonar will require for a maximum
// distance it can return.
volatile uint32_t numOV_max_sonar = 0;

// Port ping setup
volatile uint8_t *ddr_sonar = &DDRD;
volatile uint8_t *port_sonar = &PORTD;
volatile uint8_t *pin_sonar = &PIND;

// My TOP value for Fast PWM Top mode.
volatile uint8_t myTOP;

// Velocity of sound.
float vel_sound = 343.0f;

// Function Prototypes

```

Apr 22, 26 13:19

main.cpp

Page 2/7

```

// Debugging USART functions
void usart_debugging(void);
void usart_init(float baud_rate);
void usart_flush(void);
void usart_read_string(char *ptr_to_str);
void usart_send_byte(unsigned char data);
void usart_send_string(const char *ptr_to_str);
void usart_send_num(float num, char num_int, char num_decimal);

float linear_mapping(float x, float x1, float x2, float y1, float y2);
float sonar(void);
void my_delay_us(unsigned long x);
void config_tc0();

// Timer overflow, number of overflows for the current PWM signal. A
// flag gets set in hardware then the ISR gets called.
//
ISR(TIMERO_OVF_vect)
{
    numOV++;
}

void config_tc0()
{
    // One clock cycle time.
    clock_tc0 = 16.0e6 / prescaler;

    // Mode 7: Fast PWM where TOP = OCR0A
    // WGM02=1 (in TCCR0B), WGM01=1, WGM00=1 (TCCR0A)
    // COM0B1=1 (Non-inverting PWM on OC0B/PD5)
    TCCR0A = (1 << COM0B1) | (1 << WGM01) | (1 << WGM00);
    TCCR0B = (1 << WGM02); // Prescaler is 0 right now, started by
                          // start_tc0 macro.

    // Setting TOP for Fast FWM Top Mode of TC0. From a positive
    // linear mapping of last four digits of my student id card to TOP
    // value range.
    //
    myTOP = (uint8_t)linear_mapping(7723, 0, 9999, 100, 255);
    usart_send_string("myTOP=");
    usart_send_num(myTOP, 4, 0);
    usart_send_byte('\n');

    OCR0A = myTOP; // This sets the Period/Frequency of the PWM signal.
    OCR0B = 10;    // This sets the Duty Cycle (Brightness of the LED)

    // Tov=(TOP+1) // +1 a clock_tc0. With TOP=255 and 16MHz, Tov =
    // 16us
    // SUMMARY:
    // ///////////////////////////////////////////////////////////////////
    //
    // Tov = ((float)MY_TOP + 1.0f) / clock_tc0 * 1.0e6;
    //
    // Simply, it calculates how many microseconds it takes for the
    // timer to loop once.
    //

```

Apr 22, 26 13:19

main.cpp

Page 3/7

```

// MY_TOP is the number of clock ticks for the counter TC0 to go
// from 0 to MY_TOP and back to 0 again.
//
// Thus, (total_number_of_ticks * T(period of each tick)),
// is the time it take for the counter to do one cycle, thus go
// from 0ticks to MY_TOPticks.
//
// Remember that T=1/F in seconds and F=F_CPU/prescaler, in the
// program clock_tc0 is defined as clock_tc0=F_CPU/prescaler, thus
// the equation for Tov becomes:
// Tov = ((float)MY_TOP + 1.0f) / clock_tc0 * 1.0e6 us.
//
// Also, looking at the graph of how PWM's are generated, you can
// see that Tov is also the period of the PWM signal at OC0B for
// Fast PWM Top Mode.
//
// Aside, note that at compare match means, TCNT0=OCROB/A,
// OCF0B flag is set and overflow interrupt occurs, as well as clearing OCBO
.
//
// The Result: If MY_TOP is 255 and your clock is 16MHz, Tov
// becomes 16us, and prescaler=1
//
// ie Tov=(255+1)*T=256*1/F= 256*(1/(F_CPU/Prescaler))=(256*Prescaler)/F_CPU
// (256*1/16x10^6)seconds=(256/16*10^6)*1*10^-6=256/16=16us.
//
// SUMMARY: //////////////////////////////////////
Tov = ((float)myTOP + 1.0f) / clock_tc0 * 1.0e6;

// Velocity = distance/time
numOV_max_sonar = 12.0 / vel_sound / Tov * 1.0e6;
}

int main(void)
{
    usart_init(9600);

    bitSet(DDRD, pin_oc0b); // PD5 as output
    bitSet(*ddr_sonar, pin_trigger); // pin_trigger output
    bitClear(*ddr_sonar, pin_echo); // pin_echo as input

    config_tc0(); // config TC0, timer counter 0
    start_tc0; // enable TC0
    sei();

    while (1){

        float D = sonar();

        // print out the distance on the serial monitor
        usart_send_num(D, 3, 3);
        usart_send_byte(';');

        // Map distance to OCR0B (Duty Cycle) instead of OCR0A.
        //
        // Note: y2<y1(100-255) => -ve slope for mapping, which => as
        // distance increases the duty cycle decreases and hence the
        // LED attached to OCR0B gets dimmer with increasing distance.
        //
        OCR0B = (uint8_t)linear_mapping(D, 10, 300, myTOP-1, 10);

```

Apr 22, 26 13:19

main.cpp

Page 4/7

```

        // print out the duty cycle on the serial monitor
        usart_send_num(OCR0B, 3, 0);
        usart_send_byte('\n');

        // To allow the Sonar to reset
        my_delay_us(1000UL * 1000UL); // 1s delay

    }
}

float sonar(void){

    // Trigger the sonar
    bitClear(*port_sonar, pin_trigger);
    my_delay_us(2);
    bitSet(*port_sonar, pin_trigger);
    my_delay_us(11);
    bitClear(*port_sonar, pin_trigger);

    while (!bitCheck(*pin_sonar, pin_echo))
        ;

    // We have received the echo signal, start timing it buy
    // using ISR(TIMER0_OVF_vect) to count the number of overflows.
    enable_tc0_int;
    numOV = 0;
    TCNT0 = 0;
    while (bitCheck(*pin_sonar, pin_echo) && numOV < numOV_max_sonar);

    // Echo signal has gone low, stop timing.
    disable_tc0_int;
    uint8_t tmp = TCNT0;

    // Note: Velocity = Distance/Time, so,
    // D = (time_echo_signal_is_high * velocity_of_sound/2).
    // Where D is the distance_to_the_obstacle in m.

    // Remember that the Distance value returned by the ultrasonic
    // sensor is 2D, the round trip distance of the trigger/echo
    // signals.

    // Which implies that time_echo_signal_is_high = 2D/V.
    //
    // The velocity of a sound wave is 346m/s@25degreesC.
    //

    // Also remember that echo_high_us_count is in us and the
    // formula requires seconds. So divide echo_high_us_count by
    // number of us in a second.

    float tElapse = (float)numOV * Tov + ((float)tmp / clock_tc0) * 1.0e6;

    // V=d/t => d=V*t, remember the sonar returns 2*D, round trip distance.
    float Dmm = (tElapse / 1.0e6) * vel_sound / 2.0f * 1000.0f;

    return Dmm;
}

```

Apr 22, 26 13:19

main.cpp

Page 5/7

```

void my_delay_us(unsigned long x)
{
    // Maximum number of overflow times, total number of full ticks in us.
    unsigned long numOV_max = (float)x / Tov;

    // Remainder time in clock cycles.
    unsigned char tcnt0_max = ((float)x - (float)numOV_max * Tov) / 1.0e6 * cloc
k_tc0;

    if (numOV_max > 0){
        numOV = 0;
        TCNT0 = 0;
        enable_tc0_int;
        while (numOV < numOV_max);
        disable_tc0_int;
    }

    TCNT0 = 0;
    while (TCNT0 < tcnt0_max);
}

float linear_mapping(float x, float x1, float x2, float y1, float y2){

    // Linear mapping equation: slope = (y2 -y1)/(x2-x1)
    //
    // We have y vs x
    //
    // The slope is essentially the gradient of the straight line, also
    // known as the rate of change. The rate of change in a linear
    // equation is constant. So when we use linear mapping, we apply
    // a constant rate of change between the input range and the
    // output range.

    // To get greater y more quickly you need to increase the slope by
    // increasion y2.

    // By swapping y1 and y2 the slope becomes -ve(as y2>y1) so we get
    // a swapped around effect, or -ve linear mapping.

    // Clamp input so it stays in sensor range.
    //
    // 1. Clamp the input to the lower bound
    if (x <= x1) return y1;

    // 2. Clamp the input to the upper bound
    if (x >= x2) return y2;

    // Note: You lose precision with integer division because integers
    // cannot store fractional parts. When two integers are divided,
    // the result is truncated (the decimal part is discarded).

    // 3. Perform Linear Interpolation (Lerp)
    // Formula: y = y1 + (x - x1) * (slope)
    // where slope = (y2 - y1) / (x2 - x1)
    return y1 + (x - x1) * (y2 - y1) / (x2 - x1);
}

```

Apr 22, 26 13:19

main.cpp

Page 6/7

```

}

//////////////////////////////////////
//                               USART helper functions                               //
//////////////////////////////////////
void usart_init(float baud_rate){

    // Setting the baud rate register:
    //
    // Fosc, internal Crystal Arduino r3 = 16MHz
    // ubrr0 = (Fosc/prescale factor * baud rate) - 1
    // U2Xn=0, prescale factor is 2/4/2=16
    // U2Xn=1, prescale factor is 2/4=8

    float ubrr0 = ( (F_CPU / (16.0 * baud_rate)) - 1);
    int ubrr0a = (int)ubrr0;

    // Round up if neccessary to the next integer.
    if (ubrr0 - ubrr0a >= 0.5) {
        ubrr0a = ubrr0a + 1;
    }

    // Set baud rate
    UBRR0 = ubrr0a;

    // Enable transmitter and receiver, and the Interrupt Service
    // Routine on the RX buffer.
    UCSRB = (1 << TXEN0) | (1 << RXEN0) | (1 << RXCIE0);

    // Set frame format: 8 bit data, 1 stop bit, no parity
    UCSRC = (0 << UMSEL01) | (0 << UMSEL00) | // Async mode
            (0 << UPM01) | (0 << UPM00) | // No parity
            (0 << USBS0) | // 1 stop bit
            (1 << UCSZ01) | (1 << UCSZ00); // 8-bit character

}

void usart_flush(void){
    char dummy;

    // Check if the UCSROA(USART Control and Status Register) has the
    // RXC0 flag set. If flag set then there is unread data in the
    // receive buffer. Read what's in the RX buffer which clears the
    // buffer.
    while (bitCheck(UCSR0A, RXC0))
        dummy = UDR0;

    ((void)dummy); // To suppress warning
}

void usart_send_byte(unsigned char data){

    // Send a byte when the UDRE0(USART data register empty flag) id
    // is set. In the UCSROA (UART contorl or status register A). It
    // means that the USART data register is empty.
}

```

Apr 22, 26 13:19

main.cpp

Page 7/7

```

    while (!bitCheck(UCSR0A, UDRE0))
    ;
    UDR0 = data;
}

// check if const is really needed, cgpt says c++ compiler is stricter
// than c compiler.
void usart_send_string(const char *pstr){

    // note: const means a read-only-string, prevents accidentally
    // modifying string literals(which causes crashes). String
    // literals in c are a sequence of characters enclosed in double
    // quotes, that is, an constant array of characters that are null
    // terminated, ie '\0' terminated.

    // Send each byte, one at a time, till the string terminator '\0'
    // is sent. '\0' is literally the numeric value 0 written in
    // character form(NULL ACSII character).
    while (*pstr != '\0') {
        usart_send_byte(*pstr);
        pstr++;
    }
}

void usart_send_num(float num, char num_int, char num_decimal){

    // Send a number from MCU to PC by converting it to a string.

    // dtostrf(num, width, precision, string) converts float to an
    // ASCII string. We add a string terminator '\0' so the PC knows
    // when we are at the end of a string).

    char str[20];
    if (num_decimal == 0)
        dtostrf(num, num_int, num_decimal, str);
    else
        dtostrf(num, (num_int+num_decimal+1), num_decimal, str);
    str[num_int+num_decimal+1] = '\0';
    usart_send_string(str);
}

```